

5조 최종 보고서

Q-time과 예방보전 및 확률적 장비 고장을 고려한
반도체 공정 스케줄링 알고리즘 비교 연구박성현¹, 김형훈², 김진호³, 서훈⁴, 이승민⁵¹송실대학교 산업정보시스템공학과 20192208²송실대학교 산업정보시스템공학과 20192208³송실대학교 산업정보시스템공학과 20192208⁴송실대학교 산업정보시스템공학과 20192208⁵송실대학교 산업정보시스템공학과 20192208

Abstract 본 연구는 반도체 공정 스케줄링 문제에서 QTime과 PM을 고려한 최적화 모델을 제안한다. 기존 연구에서는 주로 Makespan과 Total Tardiness를 최적화 대상으로 삼았으나, 본 연구에서는 QTime과 PM을 포함하여 보다 현실적인 스케줄링 문제를 다룬다. 제안된 모델은 Mixed Integer Linear Programming (MILP)으로 수학적으로 표현되며, 이를 통해 다양한 시나리오에서의 최적 스케줄링 솔루션을 도출한다. 실험 결과, 제안된 모델이 기존 모델에 비해 QTime과 PM 측면에서 우수한 성능을 보이는 것을 확인하였다.

참고로 위 Abstract는 예시 텍스트이니 신경 쓰지 말도록

Keywords 반도체 공정, 스케줄링, QTime, PM, 최적화 모델

1 서론

반도체 제조 공정에서는 여러 작업이 정해진 공정 순서를 따라 장비를 점유하며 처리된다. 제한된 장비를 어떤 작업에 어떤 순서로 할당하는지는 납기 준수, 장비 활용률, 생산 리드타임에 직접 영향을 주므로, 작업 순서와 장비 할당을 함께 결정하는 스케줄링 의사결정이 중요하다.

반도체 공정은 단순히 작업의 처리 순서를 정하는 문제로 보기 어렵다. 하나의 작업은 정해진 공정 순서를 따라 여러 장비를 거쳐 처리되며, 특정 공정은 복수의 대체 장비에서 수행될 수 있다. 또한 동일한 공정이라도 선택된 장비에 따라 처리 시간이 달라지므로, 작업 순서뿐 아니라 장비 할당까지 함께 결정해야 한다. 이러한 구조는 공정별 대체 장비 선택을 포함하는 유연 작업장 스케줄링(FJSP)의 특성과 맞닿아 있다 (Brandimarte, 1993).

그러나 실제 제조 환경에서는 작업 순서와 장비 할당 외에도 추가로 고려해야 할 운영 특성이 존재한다. 반도체 공정에는 한 공정이 완료된 뒤 다음 공정이 일정 시간 안에 시작되어야 하는 Q-time 제약이 존재하며, 이를 초과하면 단순한 일정 지연을 넘어 품질 저하로 이어지고 위반 정도에 따라 재작업이나 폐기가 발생할 수 있다 (Cho et al., 2014). 여기에 더해 장비는 예방보전(Preventive Maintenance, PM)으로 계획적으로 비가동되거나 예기치 못한 고장으로 중단될 수 있다 (Al-Hinai & ElMekkawy, 2011; Cassady & Kutanoglu, 2005). 따라서 반도체 공정 스케줄링은 Q-time과 같은 품질 제약, PM과 같은 계획된 비가동, 장비 고장이라는

확률적 불확실성이 동시에 작용하는 동적 의사결정 문제로 볼 수 있다.

이러한 환경에서는 전체 작업 완료 시간인 makespan만으로 스케줄링 성능을 평가하기 어렵다. 특정 작업을 빠르게 처리하더라도 다른 작업이 장시간 대기하면 Q-time violation이 증가할 수 있고, 반대로 Q-time violation을 줄이기 위해 특정 작업을 우선 처리하면 전체 완료 시간이 늘어날 수 있다. 그 결과 생산 효율성과 품질 제약 관리 사이의 균형이 필요하며, makespan과 Q-time violation을 함께 반영한 의사결정 기준이 요구된다.

이와 관련해 선행연구에서는 유연 작업장 구조, Q-time 제약, 예방보전, 장비 고장이 각각 중요한 스케줄링 요소로 다루어져 왔다 (Al-Hinai & ElMekkawy, 2011; Brandimarte, 1993; Cassady & Kutanoglu, 2005; Cho et al., 2014). 예를 들어 공정별 대체 장비 선택을 고려한 FJSP 연구가 제시되었고 (Brandimarte, 1993), 공정 간 대기시간 제약을 모델링한 연구와 생산 스케줄에 예방보전을 연계한 연구도 수행되었다 (Cassady & Kutanoglu, 2005; Cho et al., 2014). 또한 장비 고장이 존재하는 유연 작업장 스케줄링 문제도 다루어졌다 (Al-Hinai & ElMekkawy, 2011). 이들 연구는 본 연구의 문제 설정과 연결되는 주요 요소를 제공하지만, Q-time violation, 장비 고장에 따른 가용성 변화, 예방보전 의사결정을 하나의 시뮬레이션 환경에서 함께 평가하는 구조와는 차이가 있다.

따라서 본 연구는 Q-time 제약, 확률적 장비 고장, threshold 기반

예방보전 의사결정을 하나의 시뮬레이션 환경에서 함께 고려한다. 기존 접근법으로는 rule-based algorithm을 구성하고, 더 넓은 해공간을 탐색하기 위해 Genetic Algorithm을 함께 적용한다 (Pezzella et al., 2008). 이를 통해 동일한 시뮬레이션 환경에서 두 접근법의 makespan과 Q-time violation을 비교하고, 생산 완료 시간과 공정 간 시간 제약 사이의 trade-off를 분석한다.

2 문제 설정 및 시뮬레이션 구성

2.1 데이터

(작성 예정)

2.2 목적 함수

(작성 예정)

2.3 시뮬레이션 환경

본 시뮬레이션은 Python의 SimPy 라이브러리를 기반으로 한 이산 사건 시뮬레이션(discrete-event simulation)으로 구현되었다. 본 절에서는 작업, 공정 순서, 장비, 고장·수리, 예방보전(PM), Q-time 관련 입력 데이터가 시뮬레이션 환경에서 어떤 상태 변화와 사건으로 처리되는지, 그리고 그 실행 결과로부터 성능 평가 지표인 makespan과 Q-time violation이 어떻게 산출되는지 설명한다. 먼저 작업 관점의 공정 흐름을 정리하고, 이어서 장비 상태 변화와 고장·수리·PM 처리 방식, 마지막으로 Q-time violation 산출 방식을 설명한다.

2.3.1 작업 공정 흐름

figure 2.3.1은 작업의 공정 흐름을 보여준다. 모든 작업은 시점 0에 투입되고, 동일한 작업 우선순위를 가지며, 장비 간 이동 시간은 0으로 가정한다. 작업 할당 전 항상 대기 공간인 스톡어(stocker)를 거치며, 스톡어의 용량은 무한대로 가정한다. 작업이 다음 공정으로 진행할 수 있는 상태가 되었더라도 처리 가능한 장비가 즉시 사용 가능하지 않으면 작업은 스톡어에서 대기한다.

장비가 사용 가능해지면 대기 작업과 장비의 조건을 바탕으로 dispatching이 수행된다. dispatching의 진입 시점은 두 가지로 구분된다. (i) 작업이 스톡어에 도착했을 때 처리 가능한 장비가 사용 가능하면 즉시 할당하고, (ii) 장비가 유휴 상태가 되었을 때 대기 작업이 있으면 후술할 dispatching 알고리즘에 따라 작업을 선택한다.

작업이 장비에 할당되면 장비에서 직전에 처리한 작업의 유형과 현재 작업의 유형을 비교하여 setup 필요 여부를 판단한다. setup이 필요한 경우 setup을 먼저 수행한 뒤 가공을 시작하며, setup이 필요하지 않은 경우에는 바로 가공을 시작한다. setup 도중

혹은 가공 도중 장비 고장이 발생할 수 있으며, 고장 시 해당 공정은 완료되지 않은 것으로 처리된다. 그 결과 중단된 작업은 같은 공정을 다시 수행해야 하는 상태로 스톡어에 복귀하고, 이후 재할당 과정을 거친다. 고장으로 인한 중단과 재투입은 여러 회 반복될 수 있다.

모든 공정을 완료한 작업은 완료 상태로 간주되어 시뮬레이션에서 제외되며, 모든 작업이 완료되면 시뮬레이션이 종료된다. 이때 마지막 작업이 완료되어 시뮬레이션이 종료되는 시점을 makespan으로 기록한다.

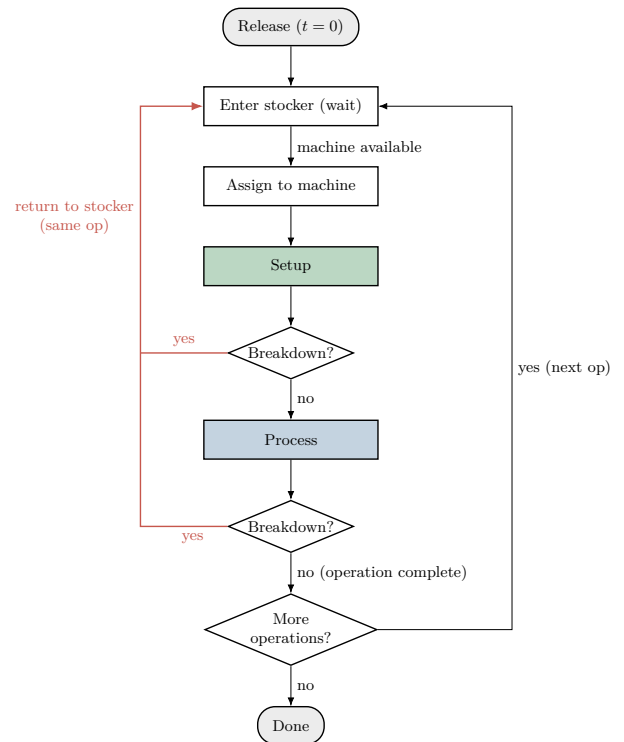


figure 2.3.1: 작업 흐름도

2.3.2 장비 상태 변화

figure 2.3.2는 장비 상태가 시간에 따라 어떻게 변하는지 예시 간트차트를 통해 보여준다. 장비는 유휴, setup, 가공, 수리, PM 수행 중 하나의 상태를 가질 수 있으며, 한 번에 하나의 작업만 처리할 수 있다.

각 장비의 고장 발생 시점은 장비별 고장 분포에서 추출한 다음 고장까지의 시간으로 결정한다. 해당 시간이 경과하면 장비는 고장 상태로 전환되며, 이는 유휴 상태뿐 아니라 setup 또는 가공 중에도 발생할 수 있다. 처리 중 고장이 발생하면 진행 중이던 공정은 중단되고, 장비는 정해진 수리 시간 동안 수리 상태를 거친 뒤 다시 사용 가능 상태로 복귀한다. 수리가 완료된 장비는 setup 판단 기준이 초기화되므로, 복귀 직후 처음 처리하는 작업에는 setup 시간이 발생하지 않는다.

예방보전(PM)은 후술할 PM 규칙에 따라 장비 고장 가능성을 낮추기 위해 수행한다. PM은 장비가 유휴 상태일 때만 수행하며, PM 예정 시점에 작업이 진행 중이면 해당 공정이 완료될 때까지 대기한 뒤 수행한다. PM 수행 중에는 고장이 발생하지 않는 것으로 간주한다. PM이 완료되면 장비는 다시 사용 가능 상태로 복귀하고, 다음 고장 발생 시점과 setup 판단 기준이 초기화된다. 따라서 수리 직후와

마찬가지로, PM 완료 후 처음 처리하는 작업에도 setup 시간이 발생하지 않는다.

2.3.3 Q-time violation

figure 2.3.3은 Q-time 누적 규칙과 Q-time violation 산출 방식을 보여준다. Q-time은 한 공정이 완료된 후 다음 공정의 가공이 정상적으로 시작되기 전까지 허용되는 최대 대기시간으로 정의하며, 작업이 이 한도를 초과하면 Q-time violation이 발생한다.

Q-time은 이전 공정 완료 후 다음 공정 대기가 시작된 시점부터 해당 공정의 정상 가공이 시작되기 전까지 누적된다. 따라서 스톡커 대기 시간과 setup 시간뿐 아니라, 고장으로 중단되어 완료되지 못한 가공 시간도 누적 대상에 포함된다. 즉 작업이 장비에 투입되었더라도 고장으로 공정이 완료되지 못하면, 그 동안 흐른 시간은 다음 공정이 아직 정상적으로 시작되지 않은 시간으로 보아 Q-time에 포함한다.

작업이 Q-time 한도를 초과하면 그 초과 시간이 Q-time violation 시간으로 기록되며, 시뮬레이션 종료 시에는 총 Q-time violation 시간을 전체 작업 수로 나눈 평균값을 산출한다.

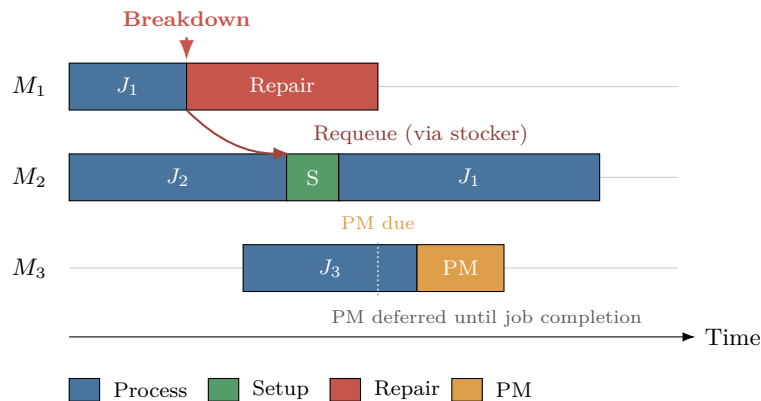


figure 2.3.2: 예시 간트 차트

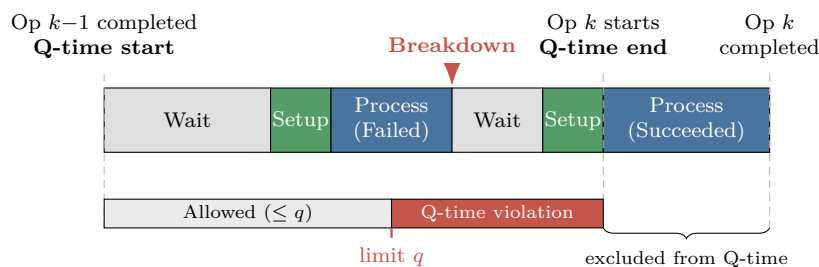


figure 2.3.3: Q-time 누적 규칙

2.4 기본 dispatching rule

(작성 예정)

2.5 Composite rule 및 PM rule

(작성 예정)

2.6 유전 알고리즘 (GA)

본 절에서는 서론에서 제시한 GA 기반 접근의 구체적인 설계를 기술한다. 제조 공정 스케줄링에서 GA는 작업 순서와 자원 할당을 하나의 해 표현으로 함께 탐색하는 방식으로 활용되어 왔으며 (Kim et al., 2021; Pezzella et al., 2008), 반도체 Q-time 제약 스케줄링에서는 염색체 인코딩과 디코딩 절차의 설계가 중요한 요소로 다루어진다 (Park & Huh, 2026). 본 연구의 GA는 작업 투입 순서, 공정별 장비 배정, 장비별 예방보전(PM) 기준을 하나의 염색체로 표현하고, 선택, 교차, 돌연변이를 반복하여 더 나은 조합을 탐색한다.

2.6.1 염색체 인코딩

본 연구의 염색체는 job_seq, machine, pm의 세 부분으로 구성된다. job_seq는 작업 투입 순서를 나타내는 순열이며, 선택 가능한 작업이 여러 개일 때 해당 순서에 따라 작업을 선택한다. machine은 각 공정이 수행될 장비를 처리 가능 장비 목록의 인덱스로 나타낸다. 따라서 하나의 공정이 여러 장비에서 처리될 수 있는 유연 작업장 구조를 염색체에 반영할 수 있다. pm은 장비별 예방보전 기준을 사전에 정의한 후보값 중 하나로 지정한다. 본 연구에서는 PM 기준 후보를 {0.05, 0.10, 0.20, 0.35, 0.50, 0.70, 0.90, 1.00}으로 두었으며, 각 값은 장비의 Weibull 누적고장확률이 해당 수준에 도달했을 때 PM을 수행하는 기준으로 사용된다.

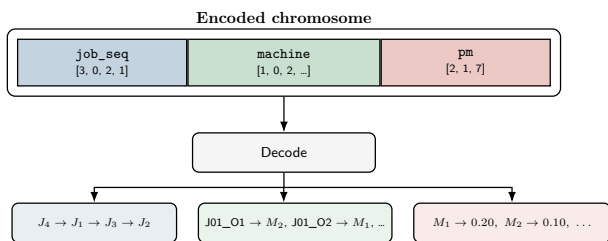


figure 2.6.1: GA 염색체 인코딩 및 디코딩 예시

figure 2.6.1은 정수 인덱스로 표현된 염색체가 디코딩을 거쳐 작업 투입 순서, 공정별 장비 배정, 장비별 PM 기준으로 변환되는 예시를 보인다. 장비 고장은 의사결정 변수로 인코딩하지 않고, 시뮬레이션 평가 과정의 상태 변화로 반영한다.

2.6.2 평가 및 세대 갱신

figure 2.6.2는 GA의 세대 갱신 과정을 나타낸다. 초기 집단은 무작위로 생성한다. job_seq는 모든 작업이 한 번씩 등장하는 순열로 생성하고, machine은 각 공정의 처리 가능 장비 중 하나를 선택하며, pm은 장비별 PM 기준 후보 중 하나를 선택한다. 생성된 염색체는 디코딩 후 시뮬레이션으로 평가하고, 그 결과를 바탕으로 해의 우열을 판단한다.

각 세대에서는 평가 결과가 우수한 상위 일부 염색체를 엘리트로 보존하여 다음 세대로 그대로 전달한다. 나머지 개체는 토너먼트 선택을 통해 부모 염색체를 선택한 뒤, 교차와 돌연변이를 거쳐 새로운 자식 염색체로 생성한다. 이후 엘리트와 자식 염색체를 합쳐 다음 세대 집단을 구성하고, 지정한 세대 수에 도달할 때까지 같은 과정을 반복한다. 이 방식은 현재까지 발견된 우수한 해를 보존하면서도 새로운 조합을 탐색하도록 한다.

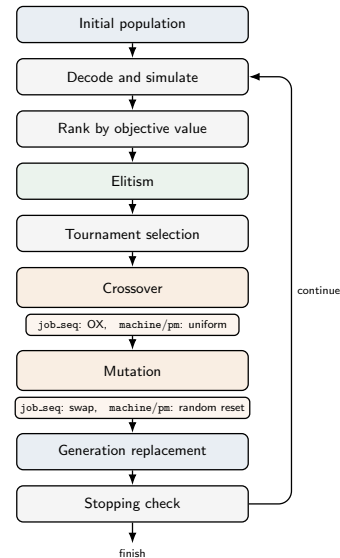


figure 2.6.2: GA 세대 갱신 흐름

2.6.3 교차와 돌연변이

교차와 돌연변이는 염색체의 세 부분이 가진 구조를 유지하도록 다르게 적용한다. figure 2.6.3와 같이 job_seq는 순열 구조를 가져야 하므로 order crossover를 사용하여 작업의 중복이나 누락이 발생하지 않도록 한다. machine과 pm은 각 위치의 값이 독립적인 인덱스이므로 uniform crossover를 적용한다. 돌연변이의 경우 job_seq에는 두 위치의 값을 바꾸는 swap mutation을 적용하고, machine과 pm에는 가능한 후보 중 기존 값과 다른 값으로 바꾸는 random reset 방식을 적용한다. 이러한 연산을 통해 모든 세대의 염색체가 실행 가능한 시뮬레이션 입력으로 유지되도록 하였다.

Crossover

job_seq: Order Crossover

P1	3	0	2	4	1
P2	1	4	0	3	2
Child	1	0	2	4	3

machine/pm: Uniform

P1	1	0	2	1
P2	0	0	1	2
Child	1	0	2	2

Mutation

job_seq: Swap

Before	3	0	2	4	1
After	3	1	2	4	0

machine/pm: Random reset

Before	1	0	2	1
After	1	0	0	1

figure 2.6.3: 교차·돌연변이 연산 예시

3 성능 평가 방법

(작성 예정)

4 결과 및 분석

(작성 예정)

5 결론

(작성 예정)

참고문헌

- Al-Hinai, N., & ElMekkawy, T. Y. (2011). Robust and stable flexible job shop scheduling with random machine breakdowns using a hybrid genetic algorithm. *International Journal of Production Economics*, 132(2), 279–291. <https://doi.org/10.1016/j.ijpe.2011.04.020>
- Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41, 157–183. <https://doi.org/10.1007/BF02023073>
- Cassady, C. R., & Kutanoglu, E. (2005). Integrating preventive maintenance planning and production scheduling for a single machine. *IEEE Transactions on Reliability*, 54(2), 304–309. <https://doi.org/10.1109/TR.2005.845967>
- Cho, L., Park, H. M., Ryan, J. K., Sharkey, T. C., Jung, C., & Pabst, D. (2014). Production scheduling with queue-time constraints: Alternative formulations. *IIE Annual Conference and Expo 2014*, 282–291.
- Kim, S., Park, S., Yoon, Y.-S., & Oh, H. (2021). A study on the scheduling of dyeing processing production based on genetic algorithm. *The Journal of Korean Institute of Communications and Information Sciences*, 46(1), 192–200. <https://doi.org/10.7840/kics.2021.46.1.192>
- Park, I. B., & Huh, J. (2026). A genetic algorithm with feasibility-agnostic encoding and three-phase decoding for scheduling semiconductor manufacturing facilities under queue time limits. *Expert Systems with Applications*, 301, 130310. <https://doi.org/10.1016/j.eswa.2025.130310>
- Pezzella, F., Morganti, G., & Ciaschetti, G. (2008). A genetic algorithm for the flexible job-shop scheduling problem. *Computers & Operations Research*, 35(10), 3202–3212. <https://doi.org/10.1016/j.cor.2007.02.014>